Universität Stuttgart

SQA

SA Project Sheet 1

DL for PIBA deliverable: 12.11.2026 EOD

Software Architecture Lab (WS 2026/27)

- **Important:** Successful completion of the course project has an impact of 40% on your final grade. The remaining percentage is the written exam.
- Participation requires you to register by joining one of the project team groups inside the *Software Architecture* ILIAS course. This means you will be part of a team responsible for developing a software project.
- **Important:** Assessment of exercise is based on the group's final presentation and not based on individual performance. Attending exercise sessions is optional but suggested while they may contain hints on improving your deliverables and overall grade.
- In this term, each team decides on its own project. You propose your project idea in a PIBA (see Task 4 of this sheet). The outcome of the project is a set of deliverables where each deliverable is specified in its own sheet. Example deliverables include but are not limited to the system's requirements (according to IEEE Std. 830-1998) or a description of the architecture.
- Each sheet guides you toward completing and submitting the deliverable. The submission of deliverables is **mandatory**.
- It is up to you to manage your team. Hint: define some roles, such as requirement engineer. Requirement engineers often should have good social skills.
- You can use *Visual Paradigm* as an enterprise suite for your project, available with the University of Stuttgart's academic license[1].
- To complete the tasks, you are advised to utilize **AI responsibly** given the provided resources for your team.

  Important: The allocated AI resources must only be used for the purpose of completing the project

[1] `https://ap.visual-paradigm.com/university-of-stuttgart2`

Task 1 Use AI Responsibly

In this project, you are encouraged to use AI tools, including the AI resources we provide for your team. You remain fully responsible for everything you submit. Read the University of Stuttgart's principles for students on using AI (*KI im Studium nutzen*)[2] before you start.

The university's principles apply to this course in full. In short:

- **Support, not substitute:** Use AI to support your own thinking, learning, and engineering work, not to replace it. You must be able to explain and defend every requirement, design decision, and line of code your team submits.
- **Responsibility for your input:** Do not enter personal, confidential, or internal data, and do not enter copyrighted material into tools that store it or use it for training. OpenRouter forwards your prompts to external model providers. Treat everything you send through OpenCode as leaving the university.
- **Responsibility for the output:** AI output can be wrong, biased, outdated, or plagiarised. Review, test, and verify everything before you use it. Generated code must be read and understood by at least one team member before it is merged.
- **Transparency:** Document which AI tools and models you used, for which tasks, and how. Every deliverable, including your PIBA, contains a short AI usage statement.
- **Fair and resource-conscious use:** Use AI where it adds real value. Prefer the university's data-protection-compliant tool RAI[3] for sensitive content.

In addition, the following course-specific rules apply:

- The allocated AI resources (API keys and budget) may **only** be used for this project. Using them for other courses, private purposes, or third parties is not allowed.
- API keys are personal. Do not share them, and never commit them to a repository.
- Misuse of the provided resources may lead to their withdrawal and is treated as a violation of the course rules.

Task 2 Familiarize with Terminology and Definitions

To use the provided AI resources effectively, you need a common vocabulary. Make sure that every team member is familiar with the key terms and concepts around LLMs and AI agents. Read the following resources:

- Building Effective Agents – Glossary[4]
- Anthropic – Building Effective Agents[5]
- Microsoft – Agent Framework, Agent Harness[6]

Task 3 AI for Software Architecture

Before you spend your budget, get an overview of where AI actually helps in software architecture (SA). Read the following systematic literature review:

- L. Schmid, T. Hey, M. Armbruster, S. Corallo, D. Fuchß, J. Keim, H. Liu, A. Koziolek: *Software Architecture Meets LLMs: A Systematic Literature Review.* arXiv:2505.16697, 2025. [7]

Use these findings to plan how you spend your budget:

- **Match the model to the task:** Use inexpensive models for well-structured, routine tasks (e.g. summarising, classifying, documentation, boilerplate code). Reserve more capable (reasoning) models for open-ended design tasks, such as deriving an architecture from requirements or analysing trade-offs.
- **Start simple:** A plain prompt with well-chosen context is often sufficient. Give the agent only the files it needs, because every token of context is billed on every request.

[2] https://www.student.uni-stuttgart.de/en/digital-services/ai-for-students/
[3] https://rai.uni-stuttgart.de/
[4] https://buildingeffectiveagents.com/glossary/
[5] https://www.anthropic.com/research/building-effective-agents
[6] https://learn.microsoft.com/en-us/agent-framework/concepts/harness?pivots=programming-language-csharp
[7] https://arxiv.org/abs/2505.16697

- **Be careful where evidence is thin:** For tasks the review identifies as barely explored, do not rely on AI output. Plan extra time for reviewing it yourself.
- **Re-evaluate during the project:** Models and prices change. Check from time to time whether a cheaper model is good enough for your recurring tasks.

Discuss in your team for which tasks of your project you intend to use AI, which kind of model you would use for each, and roughly how you will split your budget.

Task 4 Configure AI Harness

In this project, we suggest using **OpenCode** as an *AI agent harness* and provide keys for **OpenRouter** as a gateway to Large Language Models (LLMs). Follow the steps below to configure your environment.

- **Install OpenCode:** Visit the official OpenCode website (`https://opencode.ai`), download the appropriate version for your operating system, and complete the installation. OpenCode will serve as the AI agent harness for interacting with your software projects.
- **Connect OpenCode to OpenRouter:** Launch OpenCode, select OpenRouter as the model provider, and configure the connection using the provided API key. This allows OpenCode to access different LLMs for software development tasks. You can find the API key for your group in ILIAS.
- **Explore and Select LLMs:** Explore the models available through OpenRouter and compare their capabilities, performance, and pricing. Choose models according to task complexity, using inexpensive models for simple tasks and more capable reasoning models for complex software engineering problems.
- **Verify Your Setup:** Open a software project in OpenCode and ask the agent to analyze its structure or perform a small coding task. Review the generated results to verify that the environment is configured correctly.
- **Manage Your Budget:** Each group receives a **limited** OpenRouter budget, so AI usage is subject to a budget constraint. Additional budget can be requested if you provide a good reason. Monitor your credit consumption and select models carefully to minimize unnecessary spending.
- **Practice Cost-Efficient AI-Assisted Development:** Experiment with different models to understand the trade-offs between cost, speed, and output quality. The goal is to complete your SA project tasks effectively while minimizing token consumption and staying within the allocated budget.

Hint: After you successfully connected your OpenCode client, ask a free model to create a custom command `/spent` which outputs the total and remaining budget using the OpenRouter API. Make sure to you keep your API Key safe, however!

Task 5 Develop and **hand in** the PIBA Concept

Develop your own project idea as a team and write it down as a one-page PIBA, using the provided LaTeX template `PIBATemplate.tex`. Your project idea must be:

- **Feasible:** Your team can realistically implement and evaluate it within the 3 months of the course.
- **Architecturally interesting:** The challenge lies in the *architecture* (components, their interactions, quality attributes, and design trade-offs), not in the amount or difficulty of the implementation.
- **Somewhat novel:** It goes beyond re-building a standard textbook system or an existing product. It does not have to be groundbreaking.

If you are looking for ideas, you may use AI to brainstorm, following the rules in Task 1. Submit your PIBA as a PDF in ILIAS by **12.11.2026, end of day**.